

AppleSnake

Midterm Project - 2D Game Proposal
Anqi Wu

Game Name

AppleSnake. My game is based on the classic game Snake. It is most like the Google Snake version. The rules and movement come from that classic game. The look, the theme, and the name are my own. The name comes from the main idea of the game: a snake that lives to eat apples.

About the Game

AppleSnake is a single-player game made from scratch in Unity with C#. The player controls a snake that moves on a square board. The snake moves by itself at a steady speed and never stops. The arrow keys do not start or stop the snake. They only change the direction it is going. This is what makes the game fun and a little bit stressful: you cannot stop, so you have to plan ahead.

One apple shows up on a random empty square. When the snake's head reaches the apple, the player gets one point, the snake grows one piece longer, and a new apple appears somewhere else. The game ends when the snake's head hits a wall or hits its own body. Because the snake gets longer every time it eats, staying away from its own body becomes the hard part.

For the code, the snake moves on a timer using `Time.deltaTime`, so it moves at the same speed on any computer. Wall crashes use Unity colliders and tags. Hitting its own body is checked by comparing the head's next square with the list of body squares. A static, singleton `GameBehavior` class keeps track of the score, the UI, and the game state, and the score is handled with a getter/setter property. The game state is run by an enum state machine (Playing, Paused, GameOver). The game opens on a themed main menu. The Start button says "StartEat" and the Quit button says "I'm Full", which sets the fun, food theme of the whole game.

Good Outcome - What I WILL Do

These parts are already done and working:

- The snake moves on a grid at a steady speed, using `Time.deltaTime` so it is the same on every computer.
- The player controls the snake with the arrow keys. The snake cannot turn back on itself.
- An apple appears on a random empty square. Eating it makes the snake grow and moves the apple.
- Collision detection: the snake dies when it hits a wall (collider + tag) or its own body (position check).

- A point-based score shown on the screen.
- A static singleton GameBehavior manager with a getter/setter Score property (score, UI, and state in one place).
- An enum state machine (Playing, Paused, GameOver).
- A home screen with the game title, a Start button ("StartEat"), and a Quit button ("I'm Full").

Better Outcome - What I THINK I Will Do

Everything above, plus these parts, which I am working on now:

- Sound: an eat sound for the apple, a burp sound on the "I'm Full" quit button, and background music.
- One power-up made with a coroutine (for example, a special apple that speeds up, slows down, or doubles points for a few seconds).
- A pause feature that uses the Paused state in the state machine.
- A macOS build and a WebGL build, with the WebGL build put on itch.io.
-

Best Outcome - What I HOPE to Do

Everything above, plus this extra goal if I have time:

- A high-score feature. The best score is saved with PlayerPrefs, so it stays after the game closes and shows on the game-over screen and the main menu.

Next Steps and Skills

My next steps are to finish the sound (add AudioSource parts and play clips from game events and from the UI buttons), then add the pause feature, and then make the coroutine power-up. After the main features are done, I will make the macOS and WebGL builds and set up an itch.io page for the WebGL build. If I still have time, I will add the high score.

The new skills I need to learn are: Unity coroutines (how yield return and WaitForSeconds work) for the power-up; Unity sound (AudioSource and AudioClip, and how to play a one-time sound effect and looping music); PlayerPrefs for saving the high score; and how to make a macOS build and a WebGL build and put the WebGL build on itch.io. I will also use the feedback from the prototype due date to make the final version better, because this project is meant to be improved step by step.

Game Design Document

AppleSnake

Concept

A single-player arcade game. The player guides a snake around a walled board and eats apples to score. It is easy to learn but hard to master, because the snake keeps growing and never stops moving.

Gameplay

The snake moves one square at a time on a steady timer. The player changes its direction with the arrow keys. Reaching an apple gives one point, makes the snake longer, and spawns a new apple. The game keeps going until the snake hits a wall or its own body. There is no win screen; the goal is a high score.

Core Mechanics

Steady grid movement (same speed on every computer); turning with a no-reverse rule; growing when eating; random apple placement on empty squares; wall and self collision that ends the game; point-based scoring; and a state machine for playing, paused, and game over.

Control Scheme

The arrow keys turn the snake up, down, left, and right. Menu buttons are clicked with the mouse: "StartEat" to play and "I'm Full" to quit.

Story

There is no real story. The game uses a light, fun food theme instead: a hungry snake that just wants to eat as many apples as it can.

Interface

A simple Heads-Up Display shows the score with an apple icon in the corner. A "Game Over" message shows when the snake dies. The main menu shows the title and the two themed buttons.

Art and Audio

A clean, simple 2D look with a food theme: a bright snake, a plain bordered board, and a custom apple sprite. Sound adds an eat effect for apples, a funny burp on the quit button, and light background music.

Win and Lose Conditions

Lose: the snake's head hits a wall or its own body. There is no fixed win. The game is an endless high-score chase, and doing well means beating your own best score.